Object Oriented Programming (CSEG1044)
Unit 06 – Lecture 05 Notes
Swing UI Panels + Integration

Tofik Ali

February 27, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 06) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
5.1 Start with a screen/panel plan (before coding) . . . 2
5.2 Layout managers (practical cheat sheet) . . . 3
5.3 Event handling pattern (button click) . . . 3
5.4 Separation: UI vs service vs DAO (what goes where?) . . . 4
5.5 Validation + user-friendly error messages . . . 4
5.6 EDT rule + long tasks . . . 5

6 Worked Example: Add Resource → DB → refresh **5**
6.1 Service (validation + DAO call) . . . 5
6.2 List panel (JTable + refresh) . . . 6
6.3 Add dialog (short form) . . . 6

7 Deliverable-4 checklist **7**

8 In-Class Exercises (with brief solutions) **7**

9 Checkpoint Questions (with sample answers) **8**

1 Lecture Overview

In Lecture-04 you built the database schema and a working DAO layer. This lecture connects your Swing UI to your service/DAO so the application becomes a usable product.

Core goal (Deliverable-4): build one clear end-to-end flow:

$$\text{UI} \rightarrow \text{Service (rules)} \rightarrow \text{DAO (JDBC)} \rightarrow \text{DB} \rightarrow \text{UI refresh}$$

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

2 Syllabus Alignment (Unit 06)

Unit 06: Swing UI, event handling, integration with service + JDBC persistence

3 Learning Outcomes

After this lecture, you should be able to:

- Plan screens/panels and choose layout managers.
- Implement Swing event handlers that call service/DAO code cleanly.
- Validate input and show user-friendly messages.
- Complete Deliverable-4: working UI flow connected to persistence.

4 Key Terms (Quick)

- **Panel**: a `JPanel` representing a screen or a section.
- **Layout manager**: positions components (`BorderLayout`, `GridBagLayout`, ...).
- **Event**: user action like click/typing; handled by listeners (e.g., `ActionListener`).
- **EDT**: Event Dispatch Thread; Swing paints UI and runs event handlers on this thread.
- **CardLayout**: layout used to switch between screens/panels.
- **Validation**: checks that input and business rules are correct before saving.
- **Layering**: UI → service → DAO → DB.

5 Detailed Notes

5.1 Start with a screen/panel plan (before coding)

Step 1: list the screens/panels. For a Resource Booking example:

- Resource list (view/search resources)
- Add/Edit resource (short form)

- Booking form (create booking)
- Booking list (view bookings)

Step 2: decide navigation. Common patterns:

- `CardLayout` for switching pages inside one main window.
- `JTabbedPane` for independent pages.
- `JDialog` for short add/edit forms.

Step 3: pick one end-to-end use case first. For Deliverable-4, pick one:

- Add Resource → save in DB → refresh list
- Create Booking → save in DB → show booking in list

5.2 Layout managers (practical cheat sheet)

Avoid `null` layout.

Layout	Good for
`BorderLayout`	Overall frame structure (menu/top bar/content).
`FlowLayout`	Small rows of buttons.
`GridLayout`	Uniform grids.
`BoxLayout`	Vertical/horizontal stacking; simple forms.
`GridBagLayout`	Flexible aligned forms (labels + fields).
`CardLayout`	Multi-screen apps (switch screens).

5.3 Event handling pattern (button click)

Most flows follow this sequence:

1. UI reads inputs.
2. UI calls a service method.
3. UI shows success/error message.
4. UI refreshes the displayed data (table/list).

```
btnSave.addActionListener(e -> {
    try {
        service.createResource(txtName.getText(),
            txtType.getText(),
            Integer.parseInt(txtCapacity.getText()));
        JOptionPane.showMessageDialog(this, "Saved!");
        refreshTable();
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Capacity must be a number.",
            "Validation", JOptionPane.WARNING_MESSAGE);
    } catch (ValidationException ex) {
```

```
        JOptionPane.showMessageDialog(this, ex.getMessage(),
            "Validation", JOptionPane.WARNING_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage(),
            "Error", JOptionPane.ERROR_MESSAGE);
    }
});
```

Tip: keep handlers short. Put rules in service methods so you can test them.

5.4 Separation: UI vs service vs DAO (what goes where?)

UI layer:

- reads UI components and converts strings to types,
- shows messages and updates tables/lists,
- calls service methods.

Service layer:

- validates inputs and business rules,
- calls DAO methods,
- throws clear exceptions such as `ValidationException`.

DAO layer:

- performs SQL CRUD and mapping only,
- uses `PreparedStatement` and `try-with-resources`.

5.5 Validation + user-friendly error messages

Validate at two levels:

- **UI validation (fast feedback)**: empty fields, number parsing.
- **Service validation (complete)**: all rules, including cross-field rules.

Example rules:

- Resource name required (3–100 chars)
- Capacity must be > 0
- Booking start time must be before end time
- No overlapping bookings for the same resource (service rule)

5.6 EDT rule + long tasks

Swing is single-threaded:

- UI painting and event handlers run on the **EDT**.
- If you call the DB on the EDT and it is slow, the UI freezes.

If DB operations are slow, use `SwingWorker`:

```
new SwingWorker<List<Resource>, Void>() {
    protected List<Resource> doInBackground() throws Exception {
        return service.listResources();
    }
    protected void done() {
        try { updateTable(get()); }
        catch (Exception ex) { showError(ex); }
    }
}.execute();
```

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

6 Worked Example: Add Resource → DB → refresh

This worked example demonstrates the smallest complete integration. **Imports are omitted** for readability.

6.1 Service (validation + DAO call)

```
public class ValidationException extends RuntimeException {
    public ValidationException(String msg) { super(msg); }
}

public class ResourceService {
    private final ResourceDao dao;
    public ResourceService(ResourceDao dao) { this.dao = dao; }

    public void createResource(String name, String type, int capacity) throws Exception {
        if (name == null || name.trim().isEmpty()) throw new ValidationException("Name is
            required.");
        if (type == null || type.trim().isEmpty()) throw new ValidationException("Type is
            required.");
        if (capacity <= 0) throw new ValidationException("Capacity must be > 0.");
        dao.insert(new Resource(0, name.trim(), type.trim(), capacity));
    }

    public List<Resource> listResources() throws Exception {
        return dao.findAll();
    }
}
```

6.2 List panel (JTable + refresh)

```java
public class ResourceListPanel extends JPanel {
    private final ResourceService service;
    private final DefaultTableModel model =
        new DefaultTableModel(new Object[]{"ID","Name","Type","Cap"}, 0);

    public ResourceListPanel(ResourceService service) {
        this.service = service;
        setLayout(new BorderLayout());

        JTable table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);

        JButton btnAdd = new JButton("Add Resource");
        btnAdd.addActionListener(e -> openAddDialog());
        add(btnAdd, BorderLayout.SOUTH);

        refreshTable();
    }

    private void refreshTable() {
        try {
            model.setRowCount(0);
            for (Resource r : service.listResources()) {
                model.addRow(new Object[]{r.getId(), r.getName(), r.getType(), r.
                    getCapacity()});
            }
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, ex.getMessage(), "Error", JOptionPane.
                ERROR_MESSAGE);
        }
    }

    private void openAddDialog() {
        AddResourceDialog dlg = new AddResourceDialog(service, this::refreshTable);
        dlg.setVisible(true);
    }
}
```

6.3 Add dialog (short form)

```java
public class AddResourceDialog extends JDialog {
    public AddResourceDialog(ResourceService service, Runnable onSaved) {
        setTitle("Add Resource");
        setModal(true);
        setSize(420, 240);
        setLocationRelativeTo(null);

        JTextField txtName = new JTextField(20);
        JTextField txtType = new JTextField(20);
        JTextField txtCap = new JTextField(6);

        JButton btnSave = new JButton("Save");
```

```java
        btnSave.addActionListener(e -> {
            try {
                service.createResource(txtName.getText(), txtType.getText(),
                    Integer.parseInt(txtCap.getText()));
                JOptionPane.showMessageDialog(this, "Saved!");
                onSaved.run();
                dispose();
            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(this, "Capacity must be a number.",
                    "Validation", JOptionPane.WARNING_MESSAGE);
            } catch (ValidationException ex) {
                JOptionPane.showMessageDialog(this, ex.getMessage(),
                    "Validation", JOptionPane.WARNING_MESSAGE);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, ex.getMessage(),
                    "Error", JOptionPane.ERROR_MESSAGE);
            }
        });

        JPanel form = new JPanel(new GridLayout(3, 2, 8, 8));
        form.add(new JLabel("Name")); form.add(txtName);
        form.add(new JLabel("Type")); form.add(txtType);
        form.add(new JLabel("Capacity")); form.add(txtCap);

        add(form, BorderLayout.CENTER);
        add(btnSave, BorderLayout.SOUTH);
    }
}
```

7 Deliverable-4 checklist

- At least 2 UI screens/panels (list + form) with navigation.
- One complete end-to-end flow: UI → service → DAO → DB.
- Validations + user-friendly messages (no stack traces for users).
- Evidence in GitHub: issues, commits, PR; short README "how to run".

8 In-Class Exercises (with brief solutions)

1. Which use case will you implement first for end-to-end proof?

   Answer: A create flow that also lets you see data back (create + list).

2. Where should the rule "booking end time must be after start time" live?

   Answer: In service (business rule). UI may do basic date checks too.

3. Your UI freezes on "Refresh". What is the cause and fix?

   Answer: DB call on EDT; use `SwingWorker` or background thread.

4. Should DAO exceptions be shown raw to the user?

   Answer: No; show a friendly message and keep details for debugging/logs.

9 Checkpoint Questions (with sample answers)

Q1. Why separate UI/service/DAO?

Sample answer: It keeps code maintainable and testable; each layer has one responsibility.

Q2. What is the EDT and why does it matter?

Sample answer: Swing uses the EDT for painting and events; long work on EDT freezes the UI.